

AWS REST API Complete Lab

To Begin with the Lab

Summary of the Lab

In this part of the lab, a **REST API** was created in **Amazon API Gateway** to expose the backend built earlier with **AWS Lambda** and **Amazon DynamoDB**. The lab reused the existing **UsersTable** table and the three Lambda functions from the earlier **HTTP API** workflow, so the main change was switching from **HTTP API** to **REST API**. A new REST API named **UserRestAPI** was created with a regional endpoint, a TLS 1.3/1.2-compatible security policy, basic endpoint access mode, and IPv4 addressing. After that, the required resources were created so the API could support **/user** for creating a user and **/user/{userId}** for reading or deleting a specific user. The API was then deployed to a dev stage and prepared for testing through Postman.

Understanding The Lab

A **REST API** in **Amazon API Gateway** is built around resources, and each route must be created as a separate resource before methods can be attached. This is useful because the API can clearly separate collection-level actions like **/user** from item-level actions like **/user/{userId}**. In this lab, **/user** represented the full user collection, while **/user/{userId}** represented one specific record. That structure made it easy to map POST, GET, and DELETE methods to the correct **AWS Lambda** function later.

Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

API Gateway

APIs

Custom domain names
Domain name access associations
VPC links
AgentCore targets

Usage plans
API keys
Client certificates
Settings

Developer portals

Portals
Portal products

APIs (14/14)

Find APIs

Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
api-intelcore1	for testing	api-intelcore1	REST	Regional	2026-05-16	TLS_1_0	Available
APIControl1	for demo	api-control1	REST	Regional	2026-01-07	TLS_1_0	Available
APIControl2	for demo	api-control2	REST	Regional	2026-01-07	TLS_1_0	Available
apigateway2-API	Created by AWS Lambda	api-gateway2-API	REST	Regional	2025-02-19	TLS_1_0	Available
apigateway3		api-gateway3	REST	Regional	2025-02-19	TLS_1_0	Available
apigateway4		api-gateway4	REST	Regional	2025-02-19	TLS_1_0	Available
awsapigateway1-API	Created by AWS Lambda	awsapigateway1-API	REST	Regional	2025-02-19	TLS_1_0	Available
awsapigateway1-API	Created by AWS Lambda	awsapigateway1-API	REST	Regional	2026-05-16	TLS_1_0	Available
demoapi1	demoapi	demoapi1	REST	Regional	2025-04-28	TLS_1_0	Available
IntelCoreWebAPI2	for demo	intel-core-web-api2	REST	Regional	2026-05-16	TLS_1_0	Available
mydemo1		mydemo1	REST	Regional	2025-04-28	TLS_1_0	Available

Created by AWS

- Select **REST API**.
- Click **Build**.

Create API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Works with the following:
Lambda, HTTP, AWS Services

Step 2 – Create the REST API

- Select **New API**.
- Enter the API name: **UserRestAPI**
- Choose the endpoint type as **Regional**.
- Select the security policy that supports TLS 1.3 and TLS 1.2.
- Set endpoint access mode to **Basic**.

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
UserRestAPI

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new | Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
SecurityPolicy_TLS13_1_2_2021_06

Endpoint access mode - new | Info
Provide additional governance for your APIs.
☒ **Basic**
Allow clients to access the API

☐ **Strict (recommended)**
Enforce Server Name Indication (SNI) validation

IP address type | Info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

Cancel **Create API**

- Keep the IP address type as **IPv4**.
- Click **Create API**.

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new | Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
SecurityPolicy_TLS13_1_2_2021_06

Endpoint access mode - new | Info
Provide additional governance for your APIs.
☒ **Basic**
Allow clients to access the API

☐ **Strict (recommended)**
Enforce Server Name Indication (SNI) validation

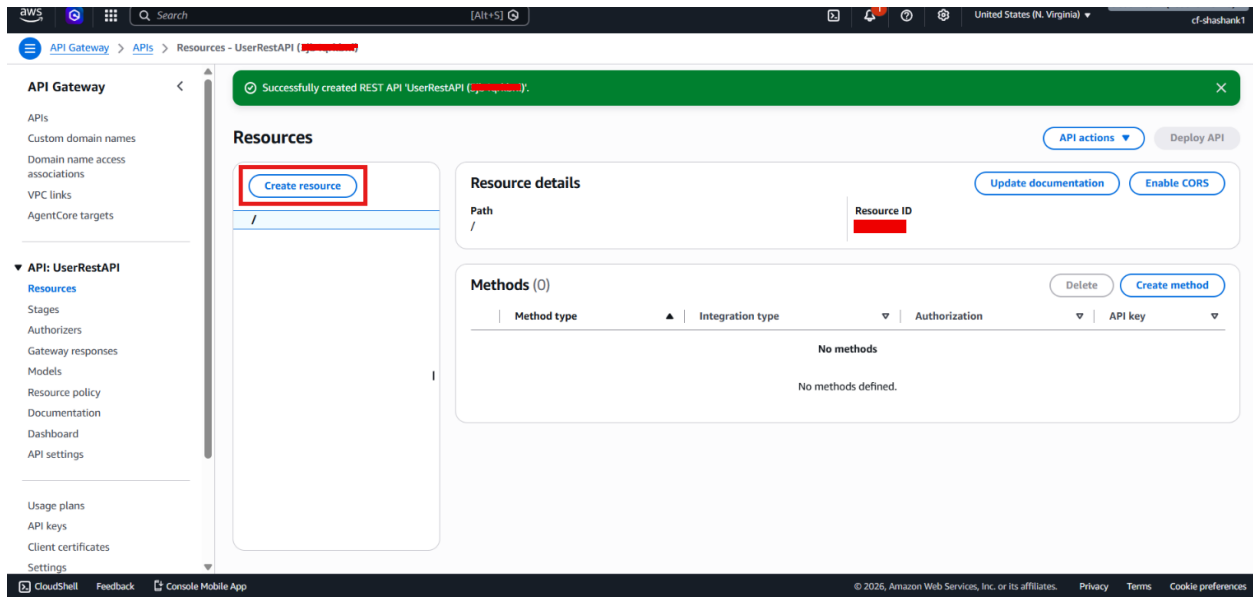
IP address type | Info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

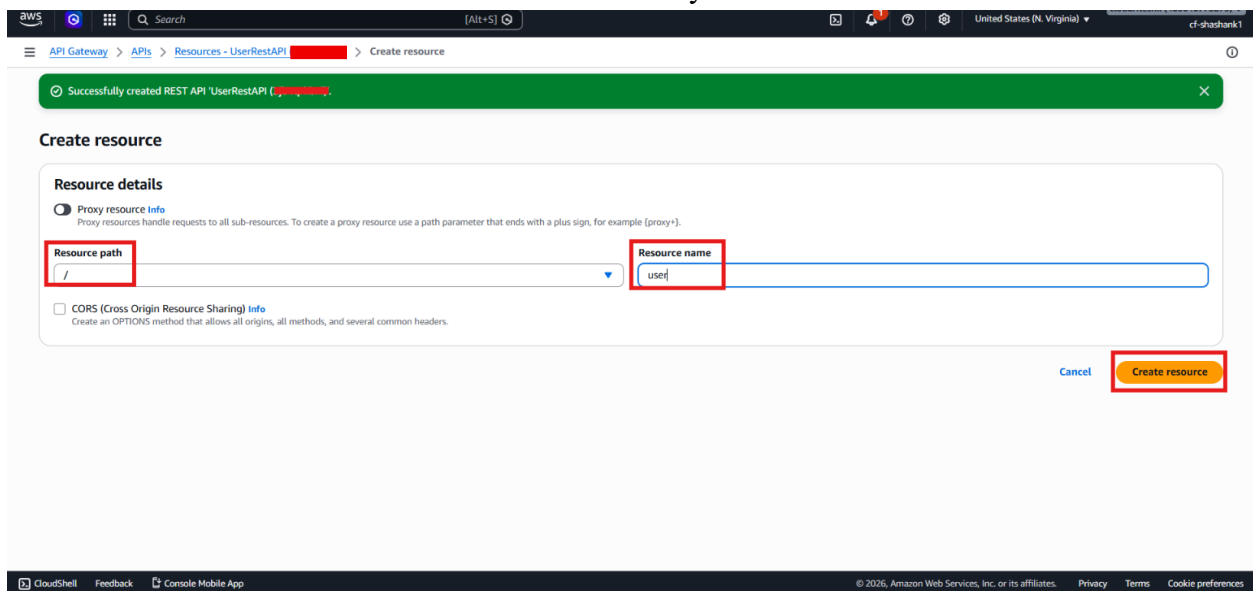
Cancel **Create API**

Step 3 – Create the Main Resource

- Open the newly created REST API.
- Click **Create Resource**.

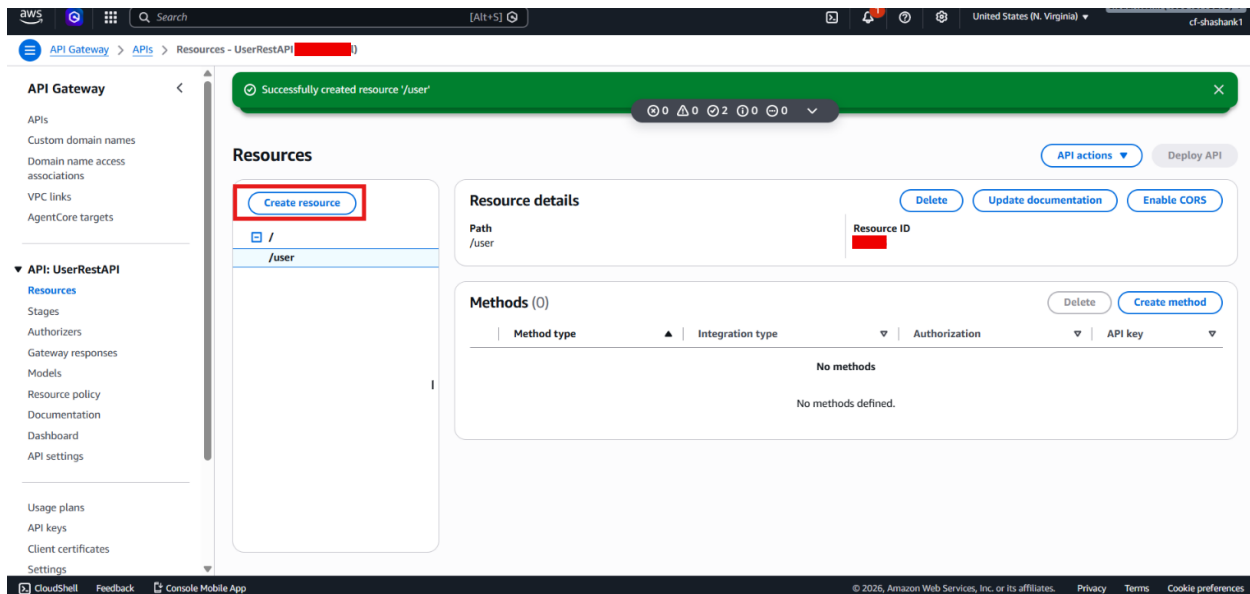


- Create the parent resource path: /user
- Confirm that the resource was created successfully.

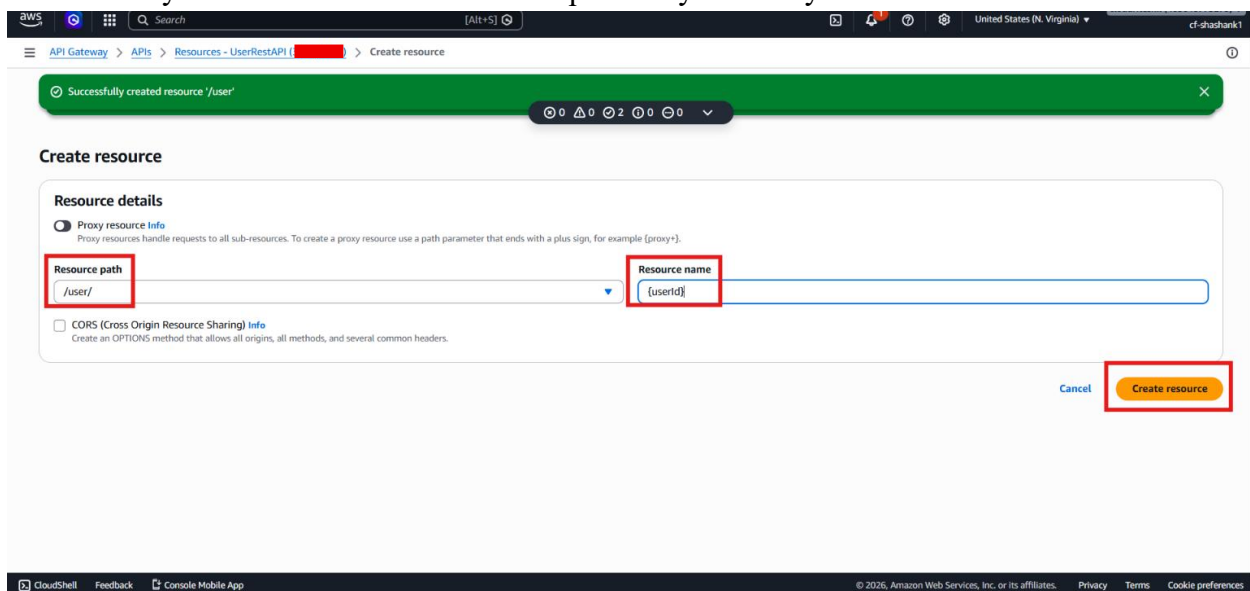


Step 4 – Create the Child Resource

- Select the /user resource.
- Click **Create Resource** again.



- Create the child resource path: `/user/{userId}`
- Use curly braces because `userId` will be passed dynamically in the URL.



- Confirm that the child resource was created successfully.

Step 5 – Prepare for Method Attachment

- Review the two resources:
 - `/user`
 - `/user/{userId}`
- Understand that each REST API URL path must be created as a separate resource.

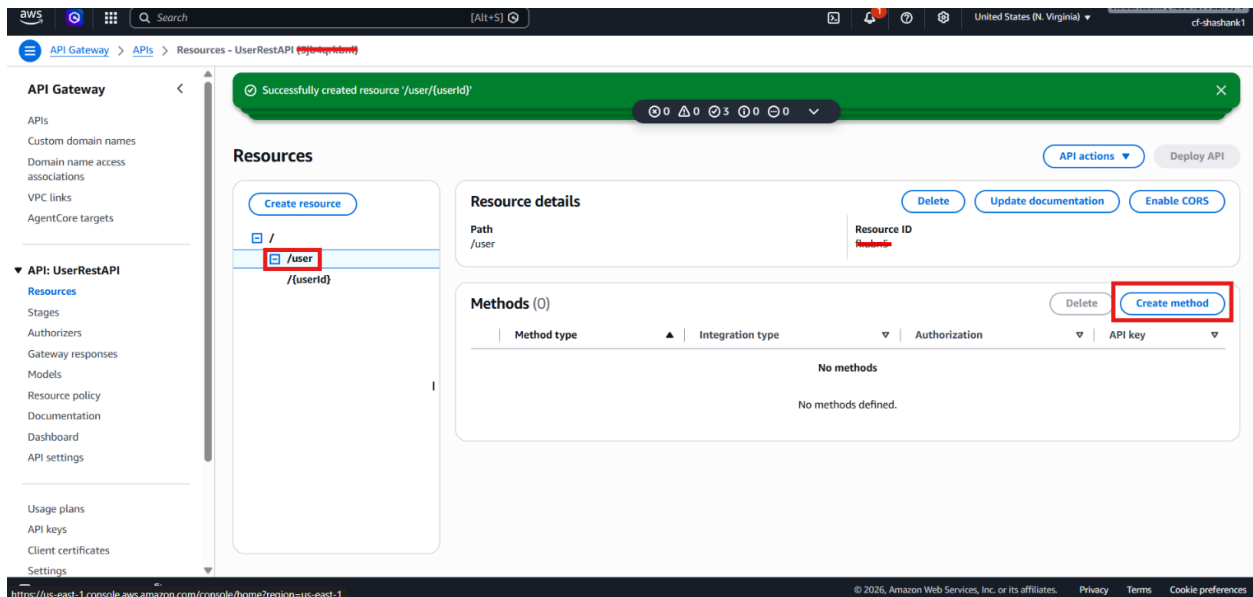
Step 6 – Open the REST API

- Sign in to the AWS Management Console.

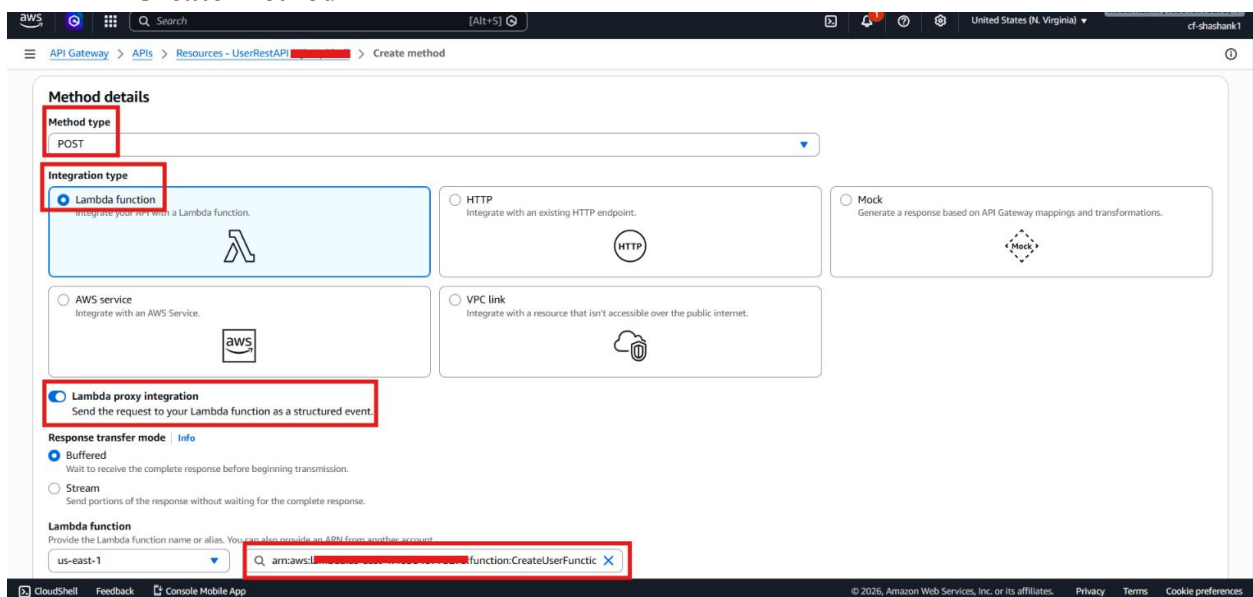
- Open **API Gateway**.
- Select the REST API created in the previous part.
- Open the **Resources** section.

Step 7 – Create the POST Method

- Select the **/user** resource.
- Click **Create Method**.

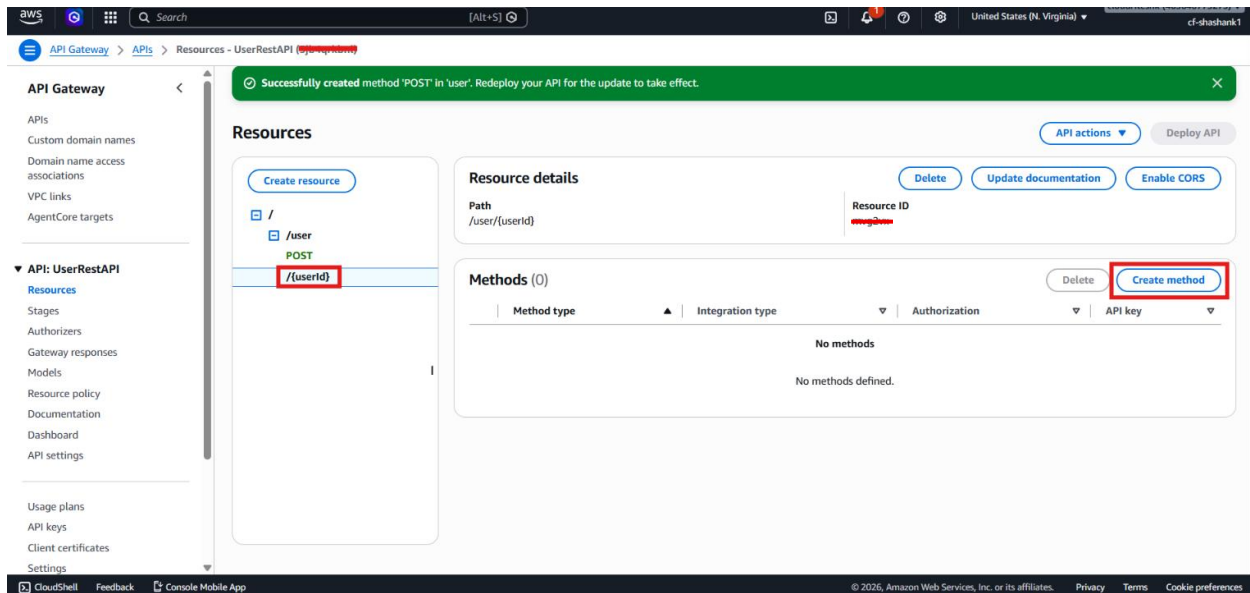


- Choose **POST**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: **CreateUserFunction**
- Click **Create Method**.

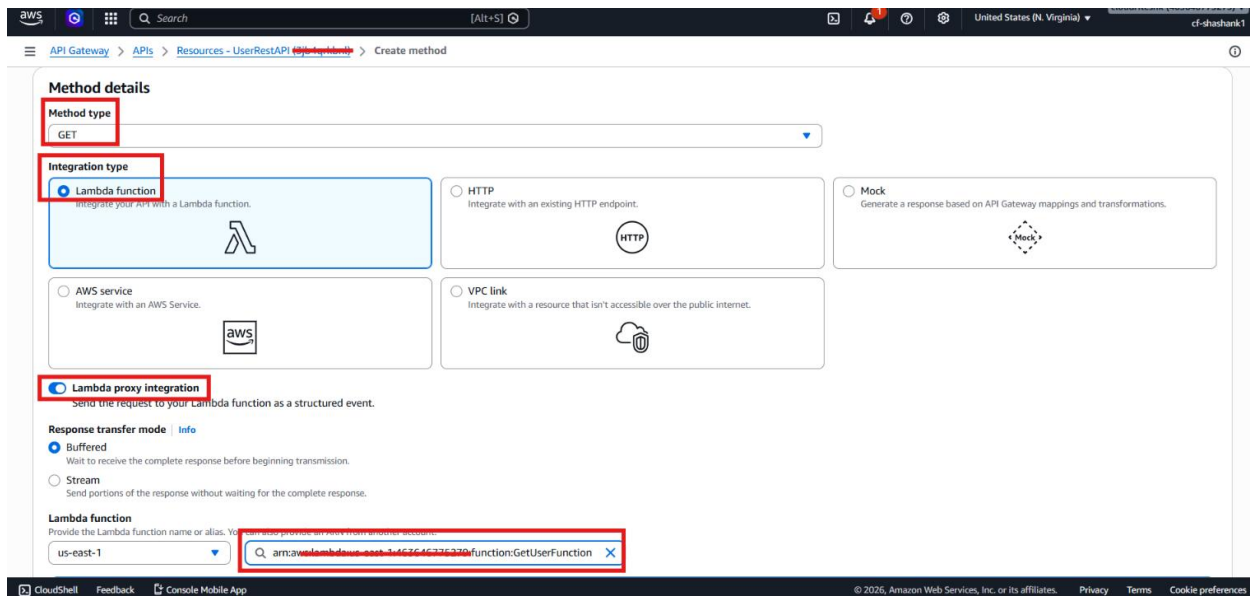


Step 8 – Create the GET Method

- Select the `/user/{userId}` resource.
- Click **Create Method**.

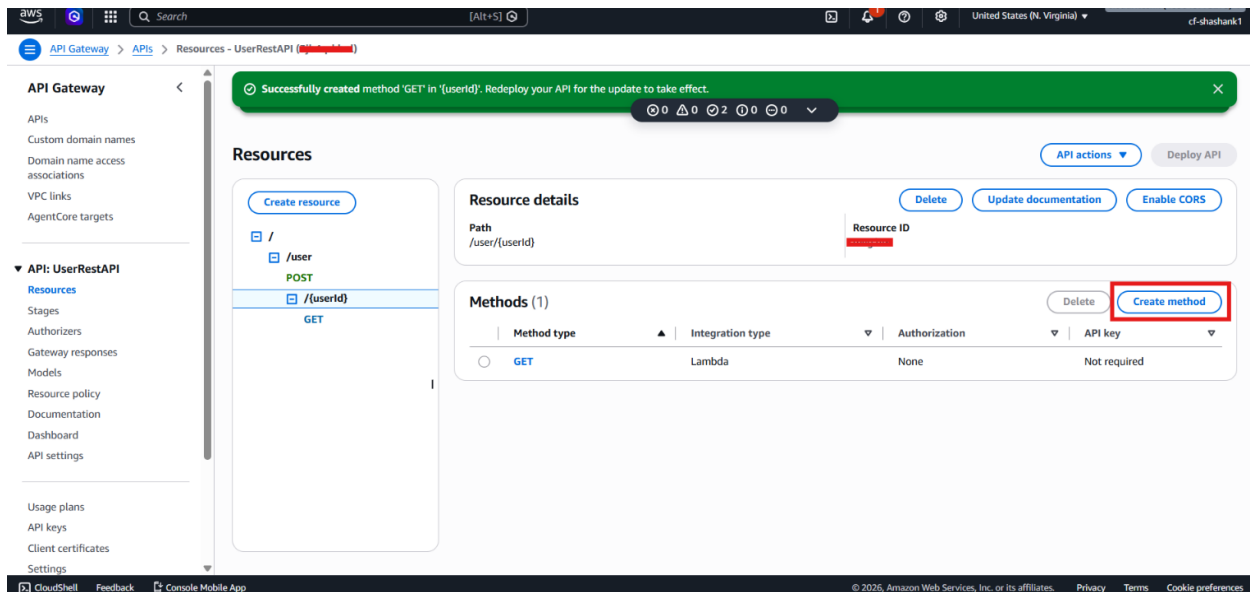


- Choose **GET**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: `getUserFunction`
- Click **Create Method**.

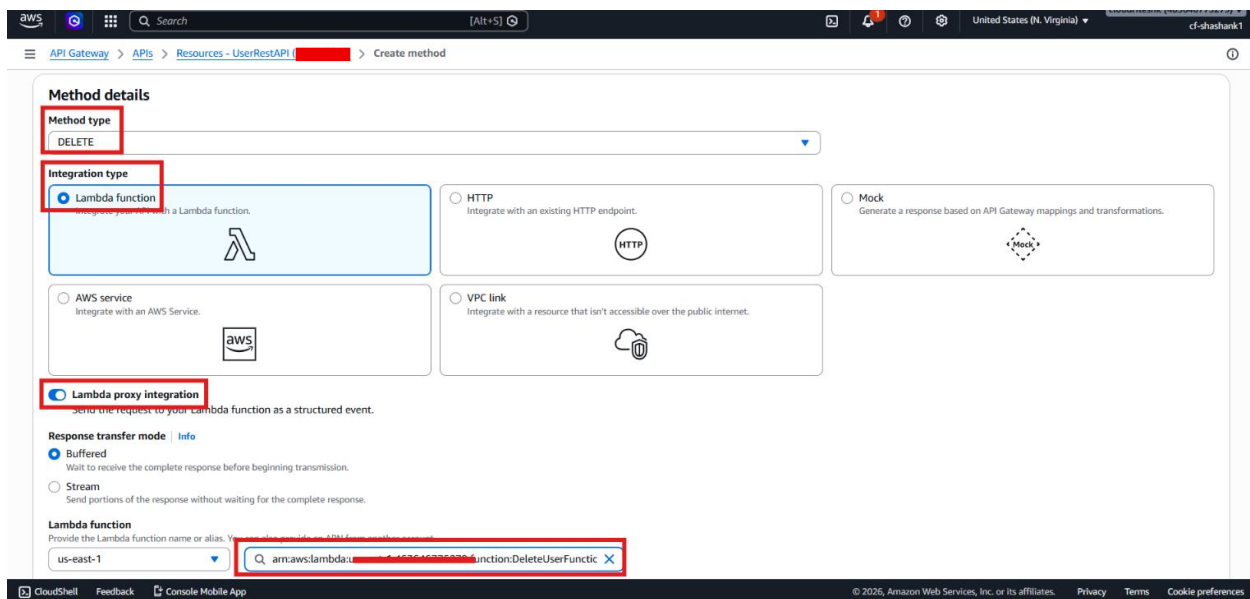


Step 9 – Create the DELETE Method

- Select the `/user/{userId}` resource.
- Click **Create Method**.



- Choose **DELETE**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: deleteUserFunction
- Click **Create Method**.

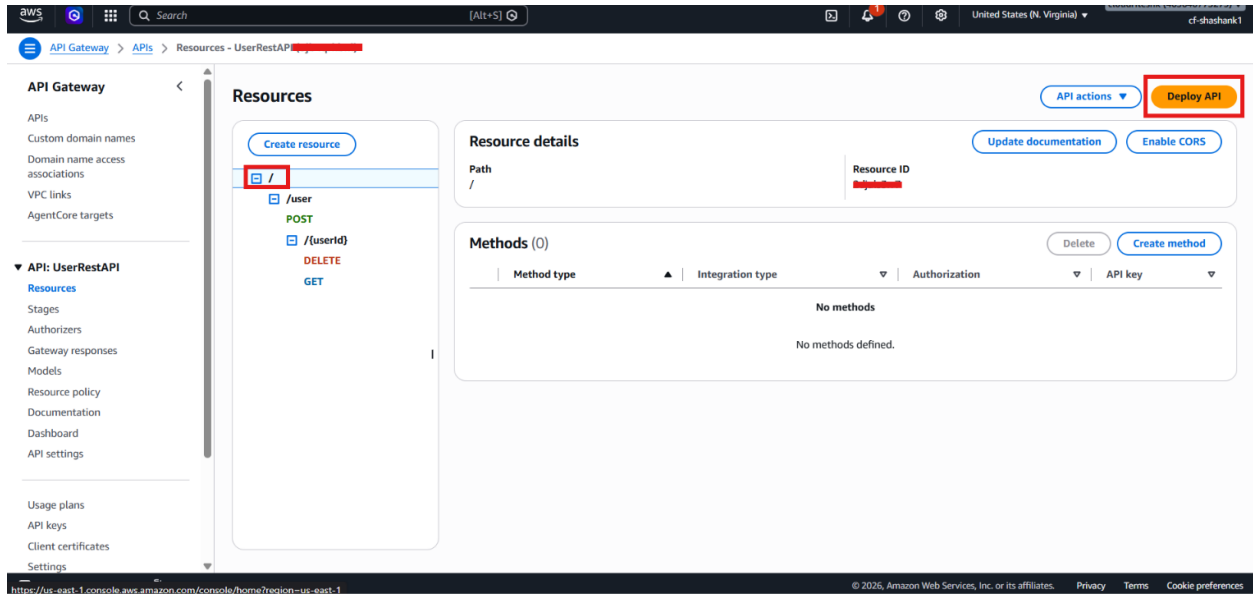


Step 10 – Review the Method Mapping

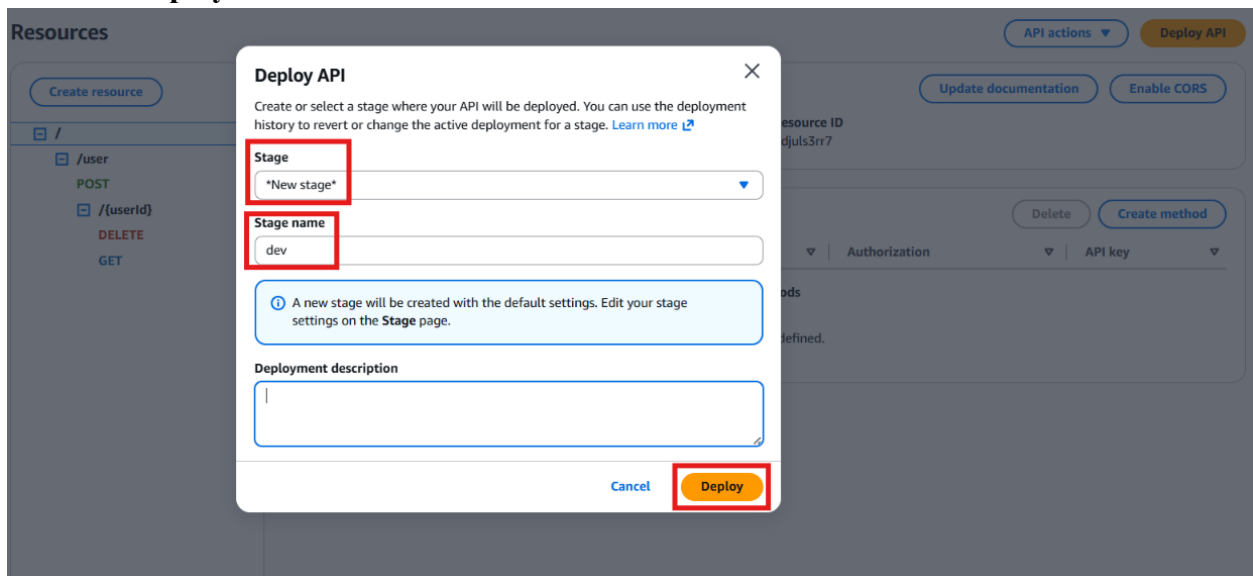
- Confirm the method-to-Lambda mapping:
 - **POST** /user → CreateUserFunction
 - **GET** /user/{userId} → getUserFunction
 - **DELETE** /user/{userId} → deleteUserFunction
- Wait for API Gateway to finish updating the API.

Step 11 – Deploy the API

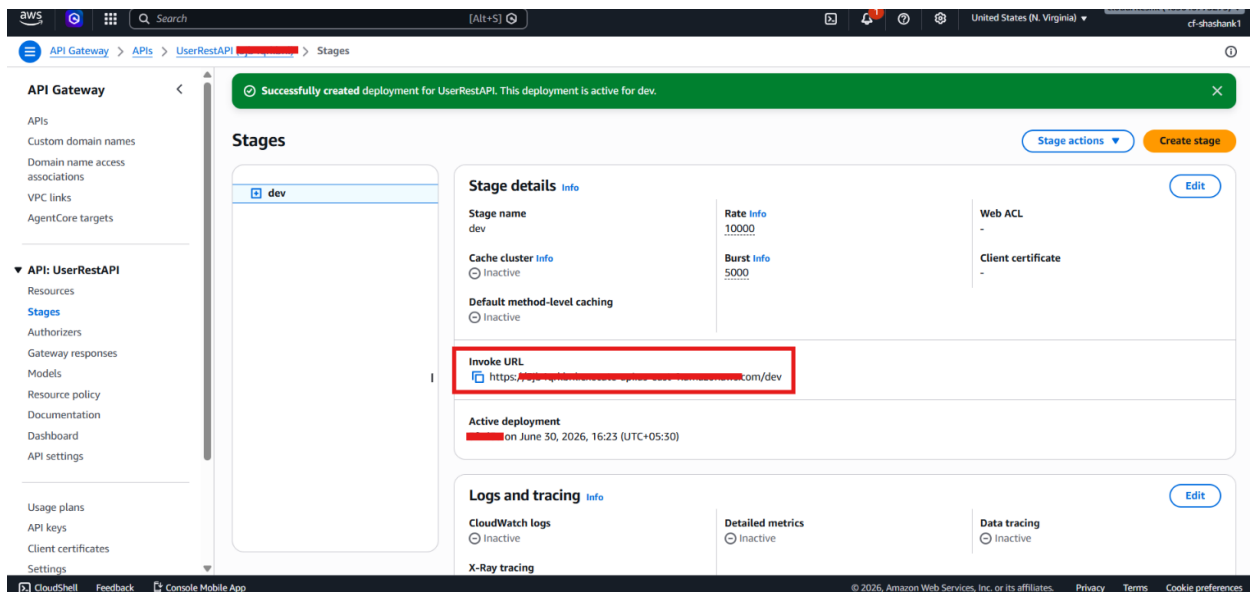
- Click **Deploy API**.



- Choose **New Stage**.
- Enter the stage name: dev
- Click **Deploy**.



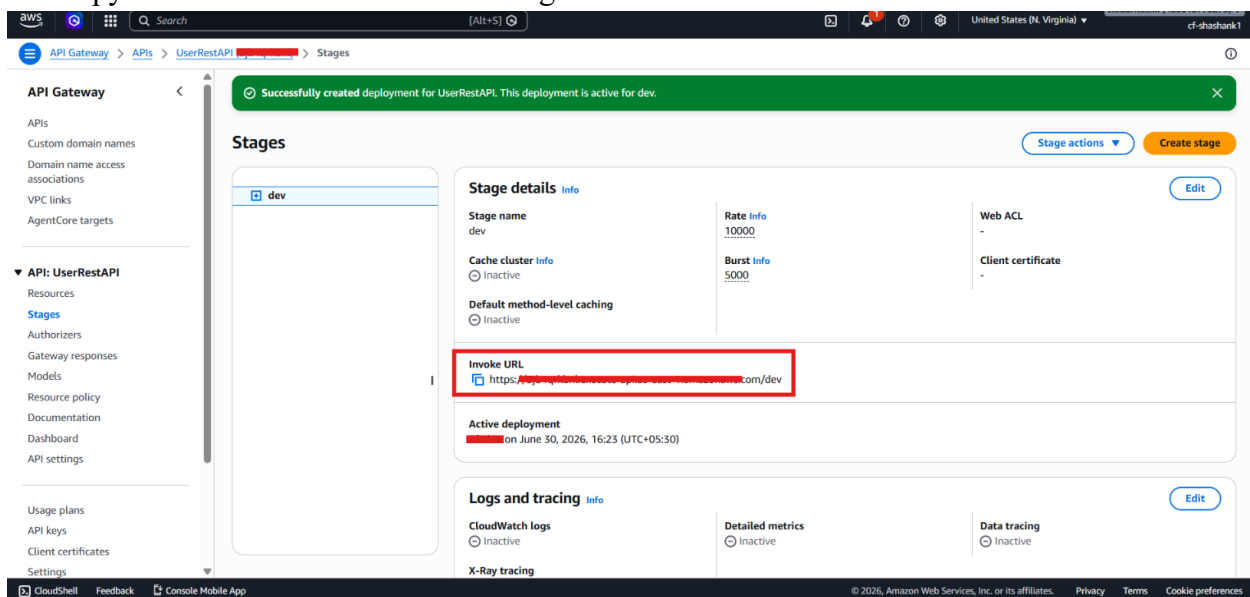
- Wait for the deployment to complete.
- Copy the generated **Invoke URL**.



- Confirm that the API is now available through the deployed stage.

Step 12 – Open the Invoke URL

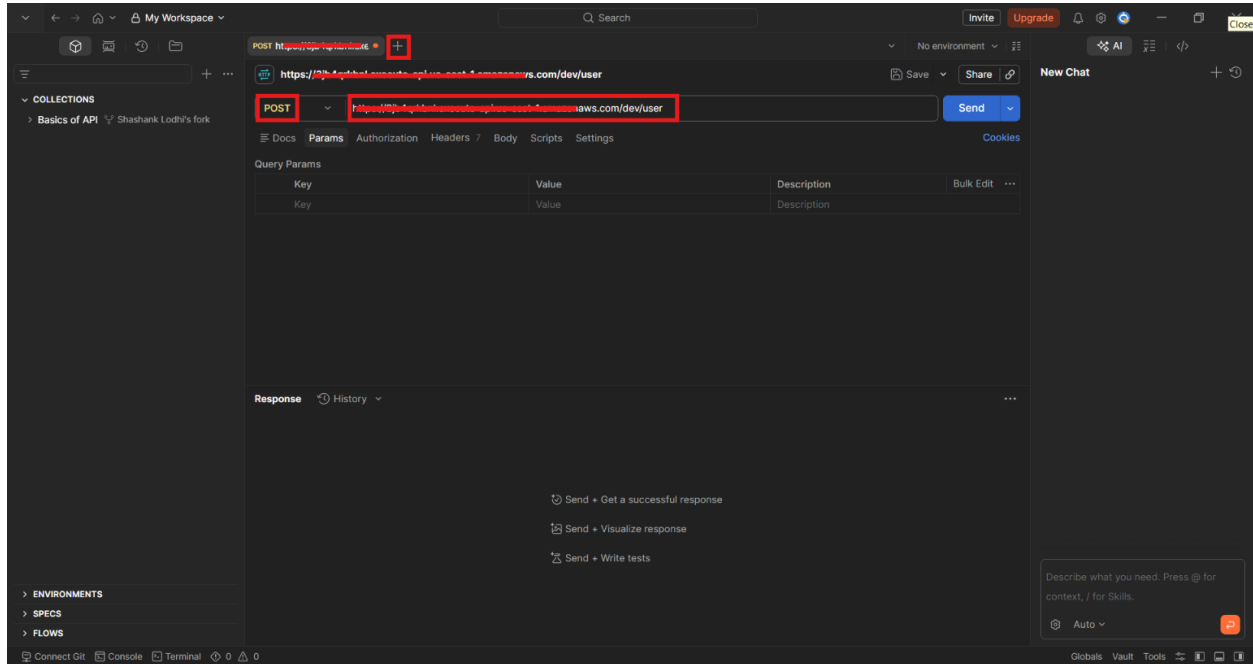
- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select the REST API.
- Open **Stages**.
- Copy the **Invoke URL** for the dev stage.



Step 13 – Test the Create User Request

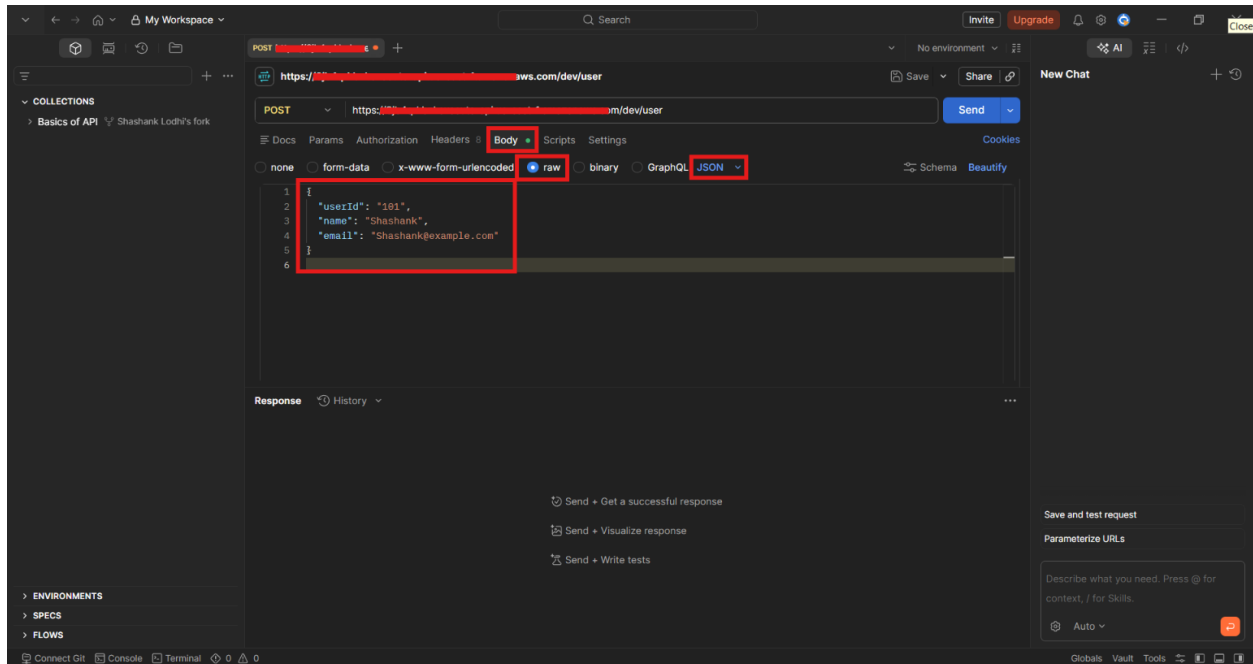
- Open **Postman**.
- Create a new HTTP request.

- Set the method to **POST**.
- Paste the invoke URL and append /user.

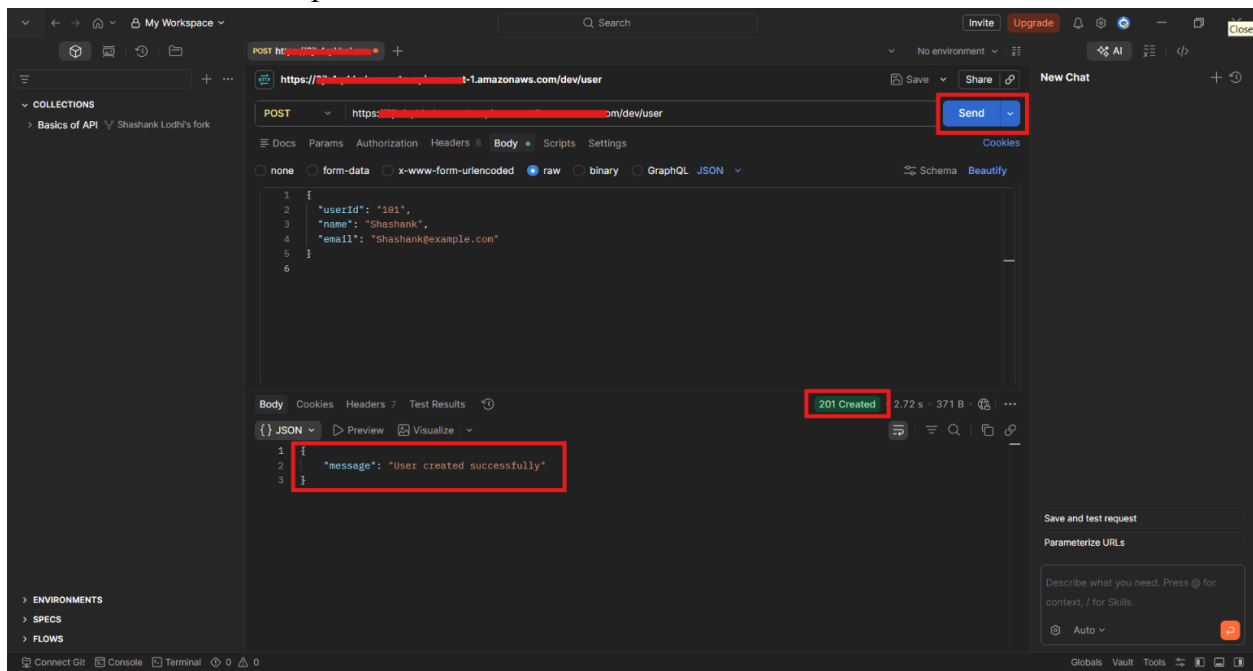


- Open the **Body** tab.
- Select **raw**.
- Choose **JSON**.
- Add the request body.

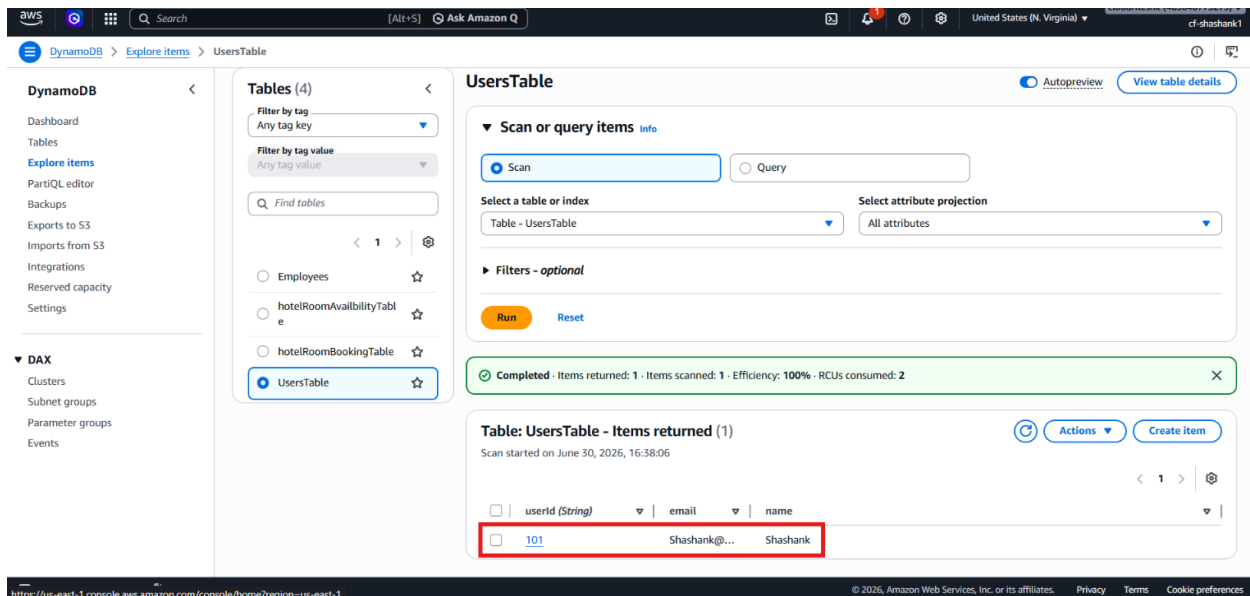
```
{  
  "userId": "101",  
  "name": "Shashank",  
  "email": "Shashank@example.com"  
}
```



- Click **Send**.
- Confirm that the response shows user creation success.



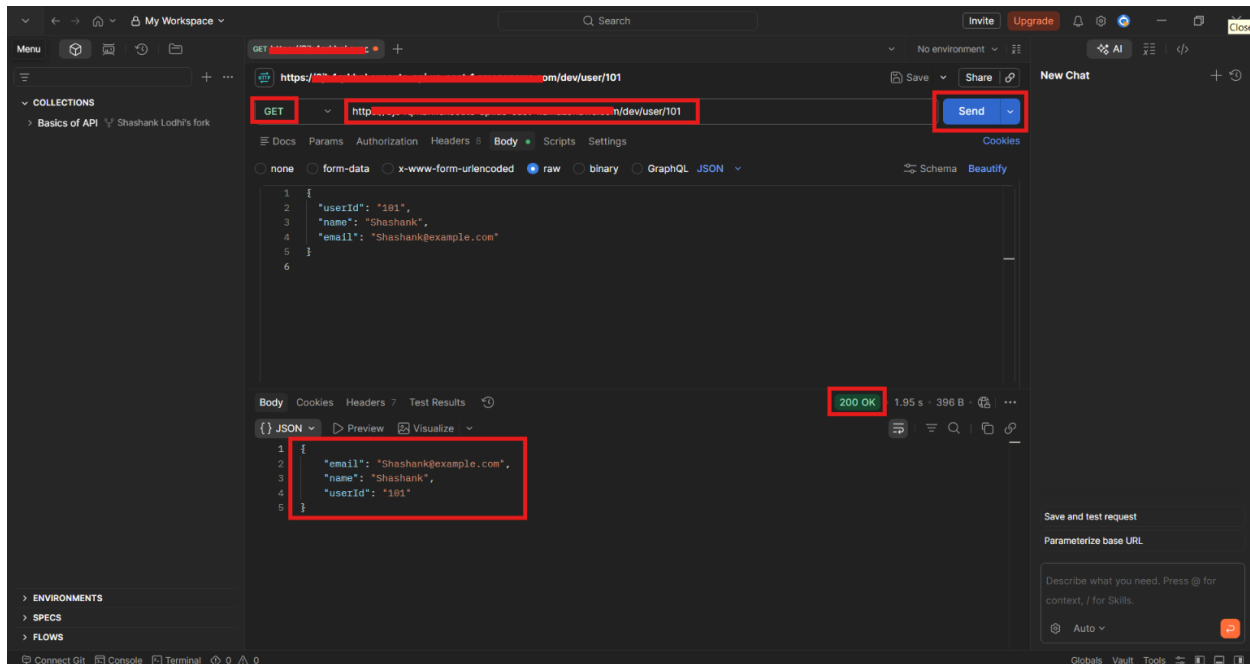
- Open **Amazon DynamoDB**.
- Go to **Tables** → **UsersTable**.
- Click **Explore table items**.



- Confirm that the new record was inserted.

Step 14 – Test the Get User Request

- In **Postman**, change the method to **GET**.
- Use the path `/user/101`.
- Click **Send**.
- Confirm that the user record is returned.

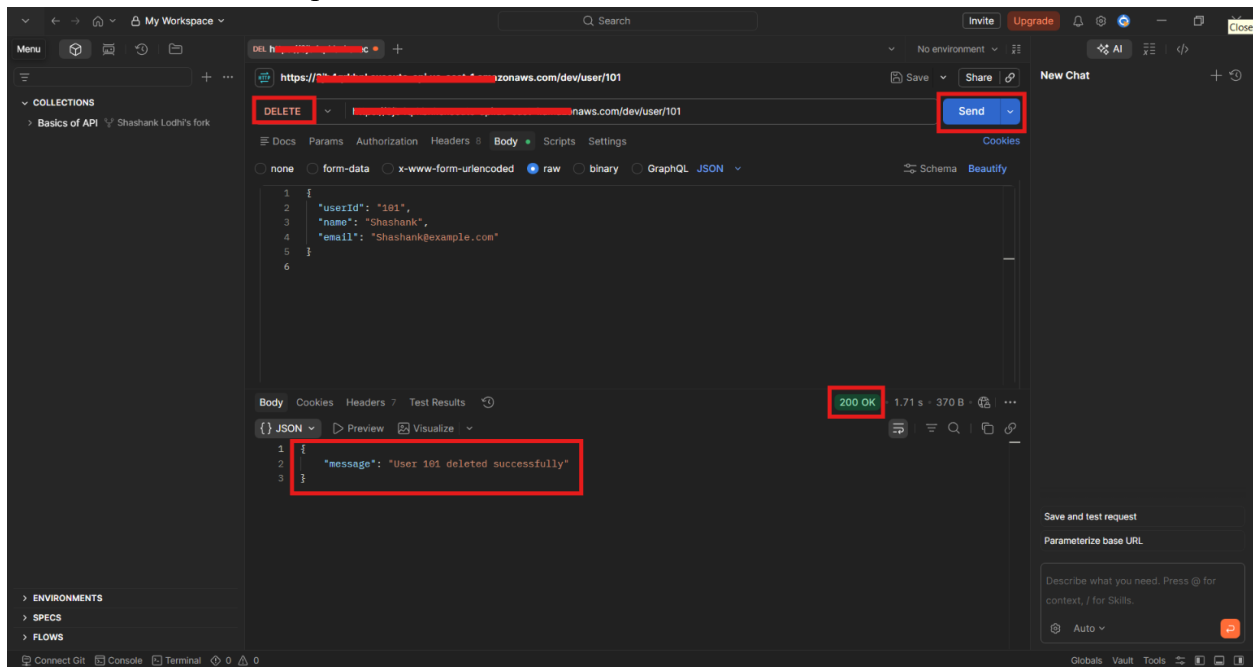


- Verify that the response matches the record stored in **UsersTable**.

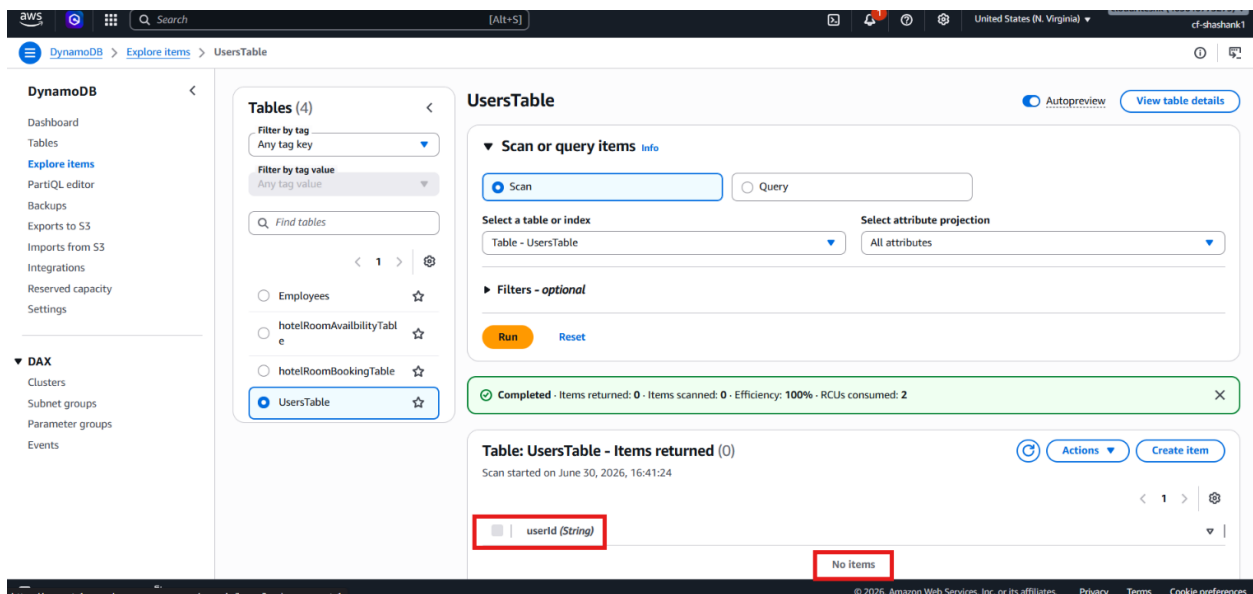
Step 15 – Test the Delete User Request

- In **Postman**, change the method to **DELETE**.

- Use the path /user/101.
- Click **Send**.
- Confirm that the response shows deletion success.



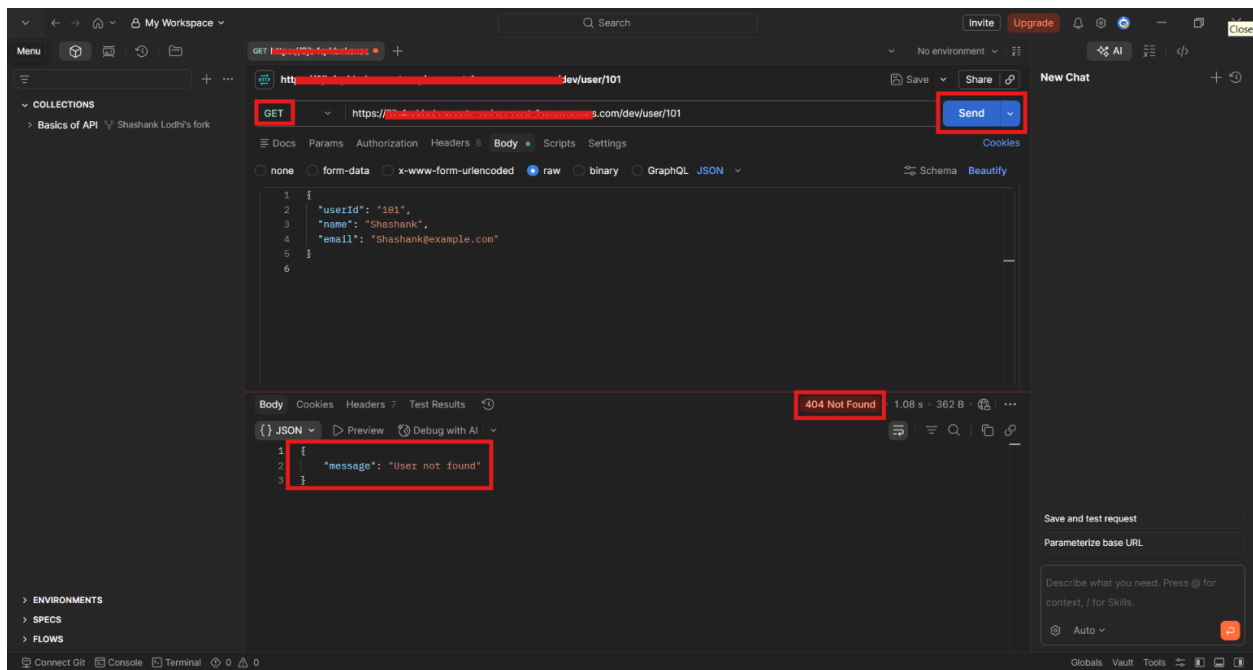
- Return to Amazon DynamoDB.
- Refresh **UsersTable**.



- Confirm that the record was removed.

Step 16 – Verify the Missing Record Case

- Send the same GET /user/101 request again after deletion.
- Confirm that the response returns **User not found**.



Verification

- Verified that a **REST API** was created successfully in **Amazon API Gateway**.
- Verified that the main resource `/user` was created.
- Verified that the child resource `/user/{userId}` was created.
- Verified that the resource structure matched the create, read, and delete use cases.
- Verified that the REST API was ready for method creation and deployment.
- Verified that the API could be tested later using the deployed dev stage invoke URL.